



A* Optimality and Heuristic Functions

Contact Session 4

Artificial and Computational Intelligence — DSE ZG557

Focus: optimality of A*, heuristic accuracy, admissible heuristics from relaxed problems, pattern databases and experience

Akhil P V | BITS Pilani WILP

Reference: T1 §§3.5.2, 3.6



BITS Pilani

Pilani Campus

From CS3 to CS4



CS3

A* was introduced as a search mechanism:

$$f(n) = g(n) + h(n)$$

CS4 asks

- “When can we trust A* to find the optimal solution?”
- “How can we design a heuristic that reduces search effort?”

A*



Correctness of h

Does the heuristic preserve optimality?

Admissibility · Consistency

+

Quality of h

How much search effort does it save?

Accuracy · Dominance

Both questions matter: a heuristic may be correct yet provide very little guidance.

Learning Objectives



By the end of the session, students should be able to:

1. Explain A* optimality.
2. Define admissible and consistent heuristics.
3. Compare heuristic quality.
4. Explain misplaced-tile and Manhattan heuristics.

5. Derive heuristics using relaxed problems.
6. Explain pattern databases.
7. Explain learning heuristics from experience.

Session focus

Objectives 1–3 concern the correctness of a heuristic; objectives 4–7 concern where good heuristics come from and how informative they are.

A* Recap



$$f(n) = g(n) + h(n)$$

$g(n)$

actual cost from the start node to n

$h(n)$

estimated remaining cost from n to a goal

$f(n)$

estimated total cost of a solution through n

Quick comparison

Algorithm	Evaluation
UCS	$g(n)$
Greedy	$h(n)$
A*	$g(n) + h(n)$

Study note

A* itself was covered in CS3. This slide is revision only — CS4 studies the properties of h .

The Real Question



$h^*(n)$ = the actual optimal cost of reaching a goal from n

$$h(n) < h^*(n)$$

Underestimate

Cautious — safe for optimality

$$h(n) = h^*(n)$$

Perfect estimate

Ideal but normally unavailable

$$h(n) > h^*(n)$$

Overestimate

May discard the optimal path

Question

“Which case can be dangerous for optimal A^* ?”

Answer

Overestimation — an inflated h can make the optimal path look worse than it is.



Admissible Heuristic

$$0 \leq h(n) \leq h^*(n)$$

Definition

An admissible heuristic never overestimates the true remaining cost of reaching a goal.

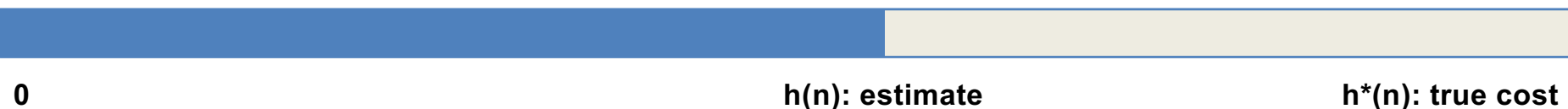
Also: $h(\text{goal}) = 0$

Memory aid

“Admissible = optimistic”

An admissible heuristic is always a lower bound on the true remaining cost.

Lower-bound view



The estimate may sit anywhere in the shaded region, but it may never pass the true remaining optimal cost.

Admissibility Check

Decide for each node whether $h(n)$ never exceeds the true remaining cost $h^*(n)$.

Node	True Cost $h^*(n)$	Estimate $h(n)$	Admissible?
A	10	7	Yes
B	8	8	Yes
C	6	9	No
Goal	0	0	Yes

Teaching interaction

Ask students to decide each row before the final column is revealed.

Why C fails

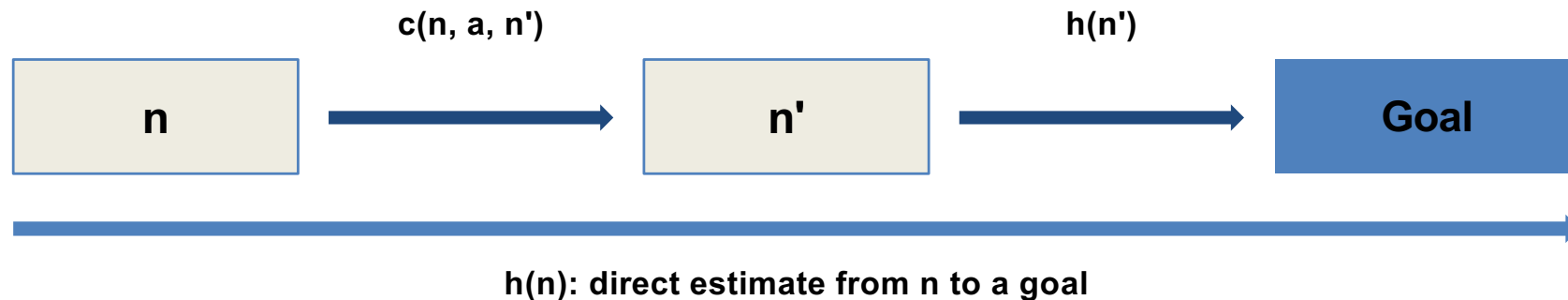
$h(C) = 9$ exceeds $h^*(C) = 6$, so the heuristic overestimates and admissibility is lost.

Equality is permitted: an estimate equal to the true remaining cost is still admissible.

Consistency (Monotonicity)

For every transition $n \rightarrow n' : h(n) \leq c(n, a, n') + h(n')$

“The heuristic estimate should obey a triangle inequality.”



$$h(n) \leq c(n, a, n') + h(n')$$

Going directly must never look dearer than stepping to a successor and estimating from there.

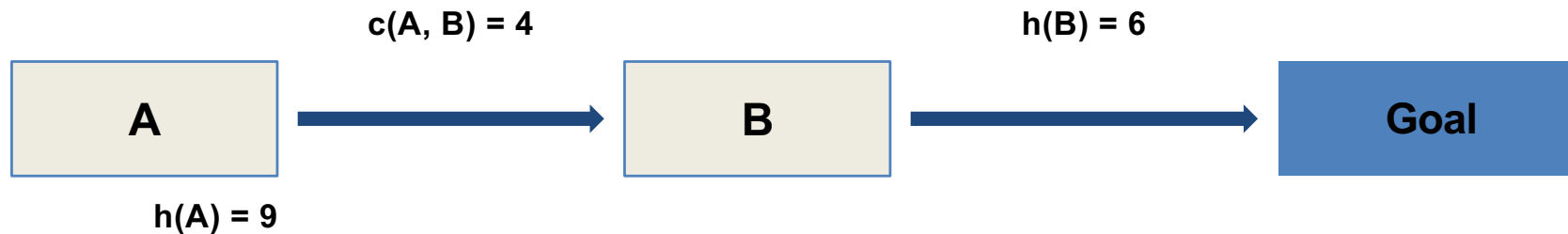
Consistency Check

Given

$$\begin{aligned}h(A) &= 9 \\c(A, B) &= 4 \\h(B) &= 6\end{aligned}$$

Check

$$\begin{aligned}h(A) &\leq c(A, B) + h(B) \\9 &\leq 4 + 6 \\9 &\leq 10 \quad \checkmark\end{aligned}$$



This edge satisfies consistency.

Important

Consistency must hold for every applicable transition, not merely for one convenient edge.



Admissible vs Consistent

Property	Meaning
Admissible	Never overestimates the true remaining cost to a goal
Consistent	Satisfies the triangle inequality on every transition

Relationship

Consistent $\Rightarrow$ Admissible

assuming $h(\text{goal}) = 0$

Admissible $\nRightarrow$ Consistent

an admissible heuristic need not be consistent

Revision points

- Consistency is the stronger property; every consistent heuristic is admissible.
- The converse does not hold in general, so do not state it as a rule.

When Is A* Optimal?

A* Version	Sufficient Condition
Tree Search	Admissible heuristic
Graph Search	Consistent heuristic

$h(n) = 0 \rightarrow A^*$ becomes Uniform-Cost Search

$h(n) = 0$ is admissible and consistent, so optimality is retained — but with no guidance at all, A* degenerates to ordering the frontier by $g(n)$.

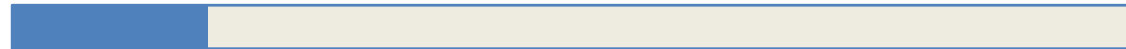
Admissible Does Not Mean Good



All three heuristics below are admissible, so all three preserve optimality.

$$h_0(n) = 0$$

No information at all — A behaves as UCS*



$$h_1(n) = \text{weak estimate}$$

Some guidance, but far below $h^(n)$*



$$h_2(n) \approx h^*(n)$$

Close to the true remaining cost — strong guidance



Same optimality guarantee — very different amounts of search.

Key takeaway

The closer an admissible heuristic is to the true remaining cost, the more guidance it usually provides.

Effective Branching Factor

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

Where

N = number of nodes generated by the search

d = depth of the solution found

b* = branching factor of a uniform tree of the same size

Interpretation

b* closer to 1 → a better heuristic

b* is a compact summary of how much search a heuristic actually caused.

High b*

→ more search

the tree spreads widely at every level

Low b*

→ less search

the search stays close to the solution path

Search effort decreases as b approaches 1*

We interpret b*; we do not solve the polynomial by hand. It is evaluated numerically from experimental data.

Dominance



If h_1 and h_2 are both admissible and $h_2(n) \geq h_1(n)$ for all n ,
then h_2 dominates h_1 .

h_1

weaker lower bound
more nodes expanded

$h_2 \geq h_1$

tighter lower bound, still admissible
fewer nodes expanded

Interpretation

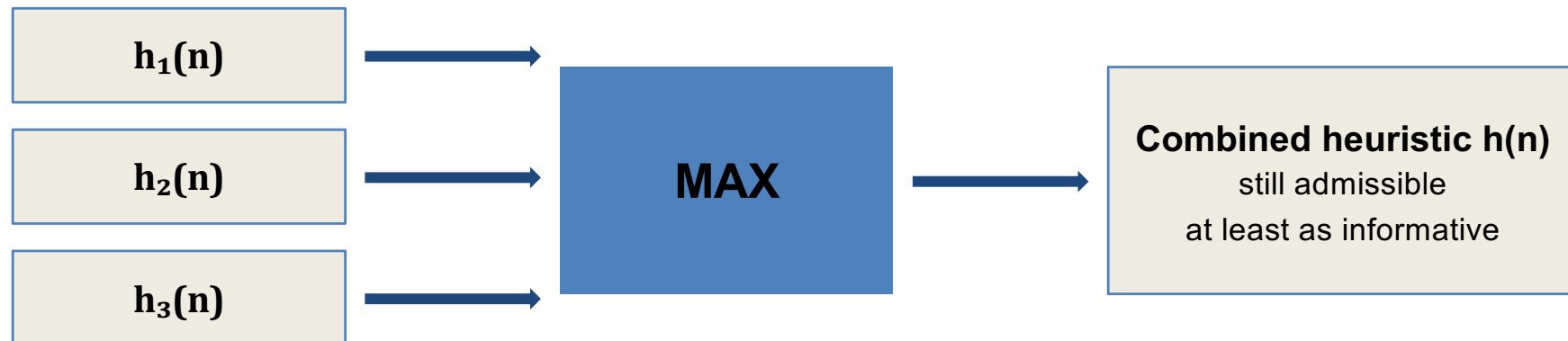
A* using h_2 normally expands no more nodes than A* using h_1 , apart from tie-related cases.

“Bigger is better — provided it remains admissible / consistent.”

Combining Admissible Heuristics

If several admissible heuristics are available and none dominates the others:

$$h(n) = \max\{ h_1(n), h_2(n), \dots, h_m(n) \}$$



Why this is safe

- Each component never overestimates, so their maximum never overestimates either.
- The maximum dominates every component, so it is at least as informative as each of them.

Choose the strongest available lower bound without losing admissibility.

The 8-Puzzle as Our Heuristic Example

Start State

7	2	4
5		6
8	3	1



Goal State

1	2	3
4	5	6
7	8	

Actions

Move the blank up, down, left or right

Path cost: 1 per move

Why this problem?

A small state space with a clear goal test, used throughout T1 to compare heuristics.

“How can we estimate the remaining cost from a given board?”

Two Admissible Heuristics

MISPLACED TILES

$h_1(n)$ = number of tiles not in their goal position
The blank is not counted as a tile.

7	2	4
5		6
8	3	1

Same board

Why admissible?

Misplaced tiles: every misplaced tile must move at least once, so the count can never exceed the true number of moves.

MANHATTAN DISTANCE

$h_2(n)$ = sum over tiles of $|\Delta\text{row}| + |\Delta\text{column}|$
The total of each tile's vertical and horizontal displacement.

Why admissible?

Manhattan distance: it ignores blocking tiles and blank-position restrictions, so it does not overestimate the movement actually required.

Tile 4 sits at row 1, column 3 but belongs at row 2, column 1: $|1-2| + |3-1| = 3$ moves. Tile 8 sits at row 3, column 1 but belongs at row 3, column 2: 1 move.

Both heuristics are admissible; Manhattan distance counts displacement, not merely wrong position.

Why Manhattan Distance Is Better

Property	Misplaced Tiles	Manhattan Distance
Measures	wrong position	distance from goal
Information	less	more
Admissible	Yes	Yes
Typical search effort	higher	lower

Empirical comparison

The textbook empirical comparison shows that Manhattan distance expands substantially fewer nodes than misplaced tiles on harder 8-puzzle cases, while both outperform uninformed search.

Manhattan distance dominates misplaced tiles: $h_2(n) \geq h_1(n)$ at every node.

Manhattan distance is still not perfect: it ignores the interaction between tiles, so it remains a lower bound rather than the true remaining cost.

Quick Activity: Manhattan Distance

Worked example — one tile

Current position: (row 1, column 3)

Goal position: (row 3, column 2)

Distance: $|1 - 3| + |3 - 2| = 3$

Your task

For the board below, calculate:

- h_1 = number of misplaced tiles
- h_2 = total Manhattan distance

1	2	3
4		6
7	5	8

Current board

Compare tile by tile
Ignore the blank.
Add $|\Delta\text{row}| + |\Delta\text{column}|$ for every
tile that is out of place.

1	2	3
4	5	6
7	8	

Goal board

Work in pairs — approximately 3 minutes. Answers will be taken from the room.

Check afterwards: is your h_2 at least as large as your h_1 ? It should be.

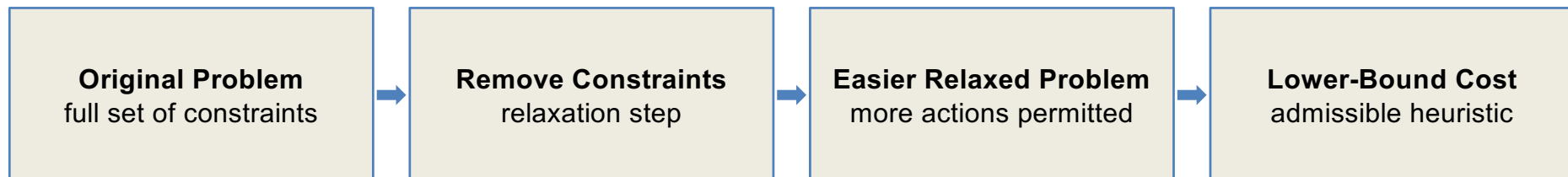
Where Do Good Heuristics Come From?

“How could we systematically invent an admissible heuristic?”

One powerful answer: RELAX THE ORIGINAL PROBLEM

Definition

A relaxed problem is obtained by removing one or more restrictions from the original problem.



Key idea

“The relaxed problem cannot be harder than the original problem.” Any solution to the original problem is also a solution to the relaxed one, so its optimal cost is a lower bound.

Manhattan distance is exactly this: relax the rule that a tile may only move into the blank square.

Why Relaxation Gives Admissibility

$C^*_{\text{relaxed}}(n)$ = optimal cost of the relaxed problem

Because the relaxed problem is easier: $C^*_{\text{relaxed}}(n) \leq C^*_{\text{original}}(n)$

Original Problem
all restrictions apply

→
Remove Constraints
relaxation

→
Relaxed Problem
easier to solve

→
Relaxed Optimal Cost
a lower bound

Derivation

Define $h(n) = C^*_{\text{relaxed}}(n)$

Then $h(n) \leq h^*(n)$

So h is admissible.

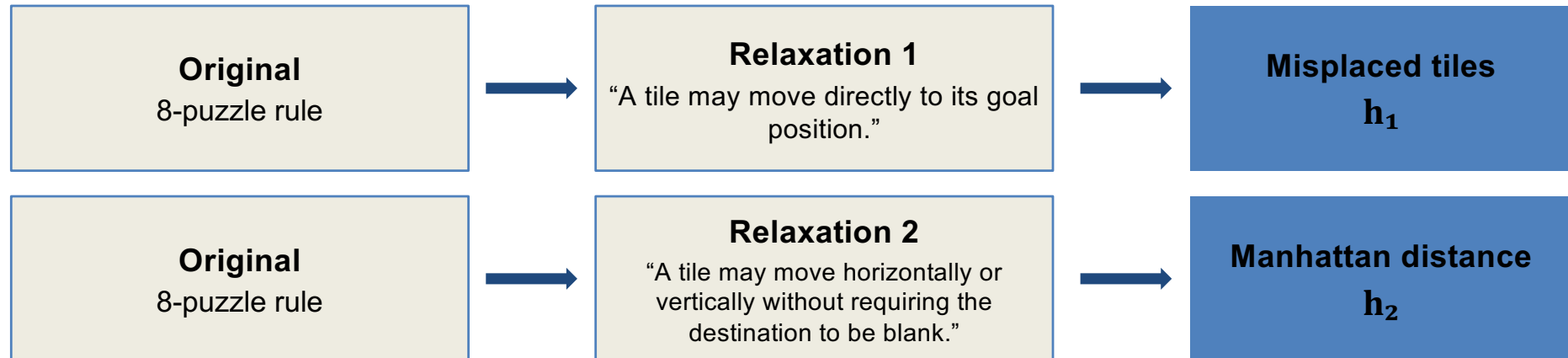
Why the inequality holds

Every solution to the original problem is still a solution to the relaxed problem, so the relaxed optimum can never be dearer.

“An optimal relaxed-problem cost is a lower bound on the original optimal cost.”

8-Puzzle Relaxations

Original rule: “A tile moves into an adjacent blank square.”



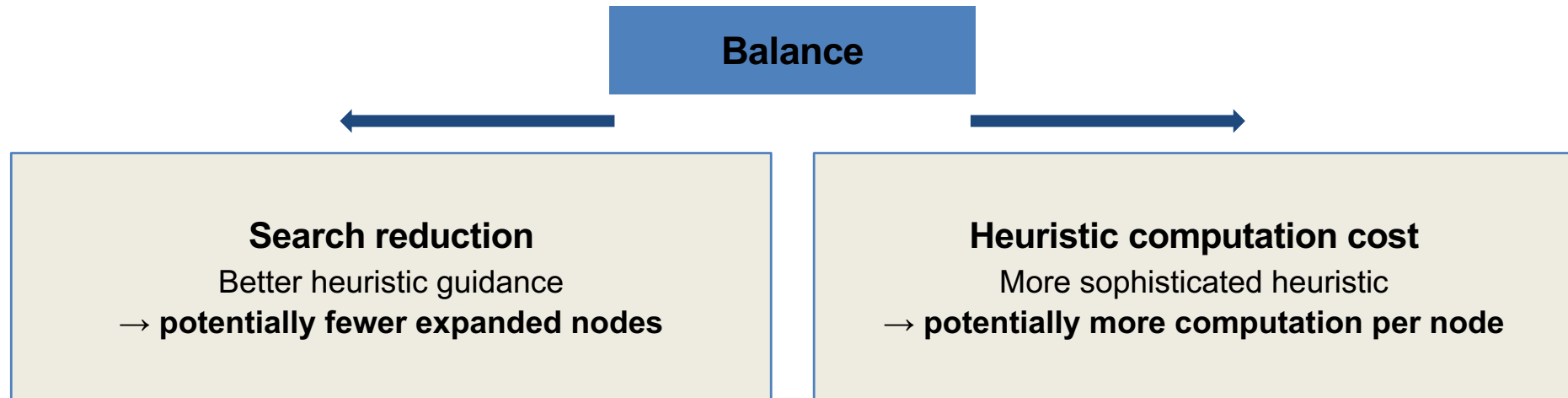
Key idea

The heuristic is obtained by solving an easier version of the original problem — not by guessing a formula.

Removing more restrictions gives an easier problem and a weaker bound; removing fewer gives a tighter, more informative one.

Accuracy vs Computation Cost

A stronger heuristic may reduce node expansion — but it may also cost more per node.



“Useful heuristic = good guidance at reasonable cost”

What to measure

Total search time, not node count alone: a heuristic that halves expansions but trebles the work per node is not a gain.

Pattern Database: Basic Idea

Choose a subproblem — for the 8-puzzle, for example, the tiles {1, 2, 3, 4}.

1	2	3
4		6
7	5	8

Pattern tiles highlighted

What is stored

Precompute the exact number of moves needed to bring the selected tiles to their goal positions, for every configuration of those tiles.

The remaining tiles are ignored — treated as interchangeable.

Selected Pattern



Pattern Database

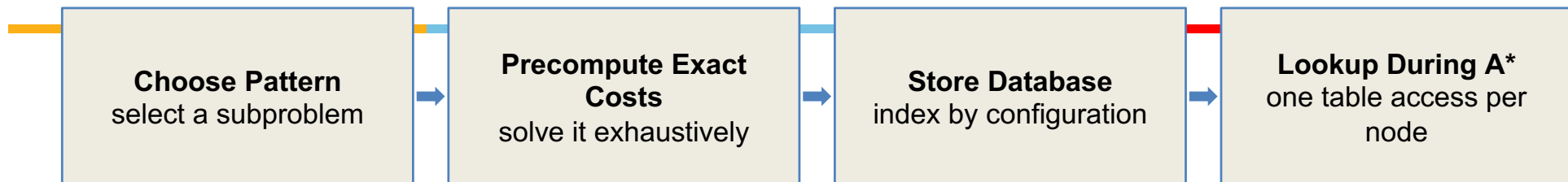


Heuristic Lookup

During A*: $h_{DB}(n)$ = lookup of the exact subproblem cost

The stored value is an exact cost for an easier subproblem, so it never overestimates the full problem — the heuristic remains admissible.

Pattern Database Workflow



Advantages

- Stronger heuristic than simple counting rules
- Fast lookup during search
- Reusable across many problem instances

Costs

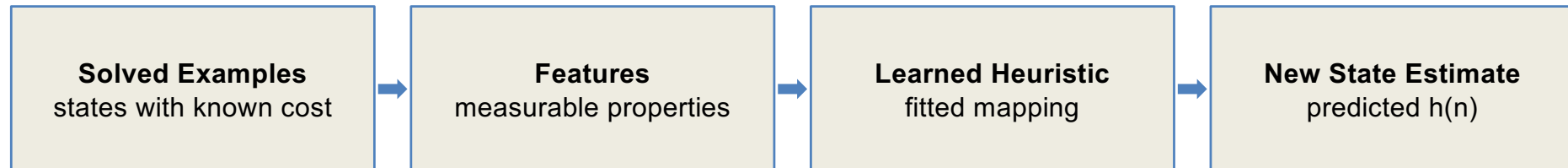
- Preprocessing time before search begins
- Memory to hold the stored table
- The pattern must be chosen with care

“Do the expensive work before search, so that heuristic evaluation during A* is fast.”

The database is built once and reused for every instance with the same goal, so the preprocessing cost is amortised.

Learning a Heuristic from Experience

Another approach: solve many examples and learn the mapping from state features to remaining cost.



Possible features

- Number of misplaced tiles
- Manhattan distance
- Number of incorrect adjacent tile pairs

Example form

$$h(n) = c_1x_1(n) + c_2x_2(n)$$

The coefficients c_1 and c_2 are fitted from the solved examples.

Use previously solved examples to estimate future problem-solving cost.

The features must be cheap to compute, otherwise the learned heuristic is too expensive to use during search.

Important Limitation of Learned Heuristics



A learned heuristic

- may be accurate
- may speed up search considerably

But
it is NOT automatically admissible or consistent.

Accurate Prediction

fits the training examples well

≠

Formal Guarantee

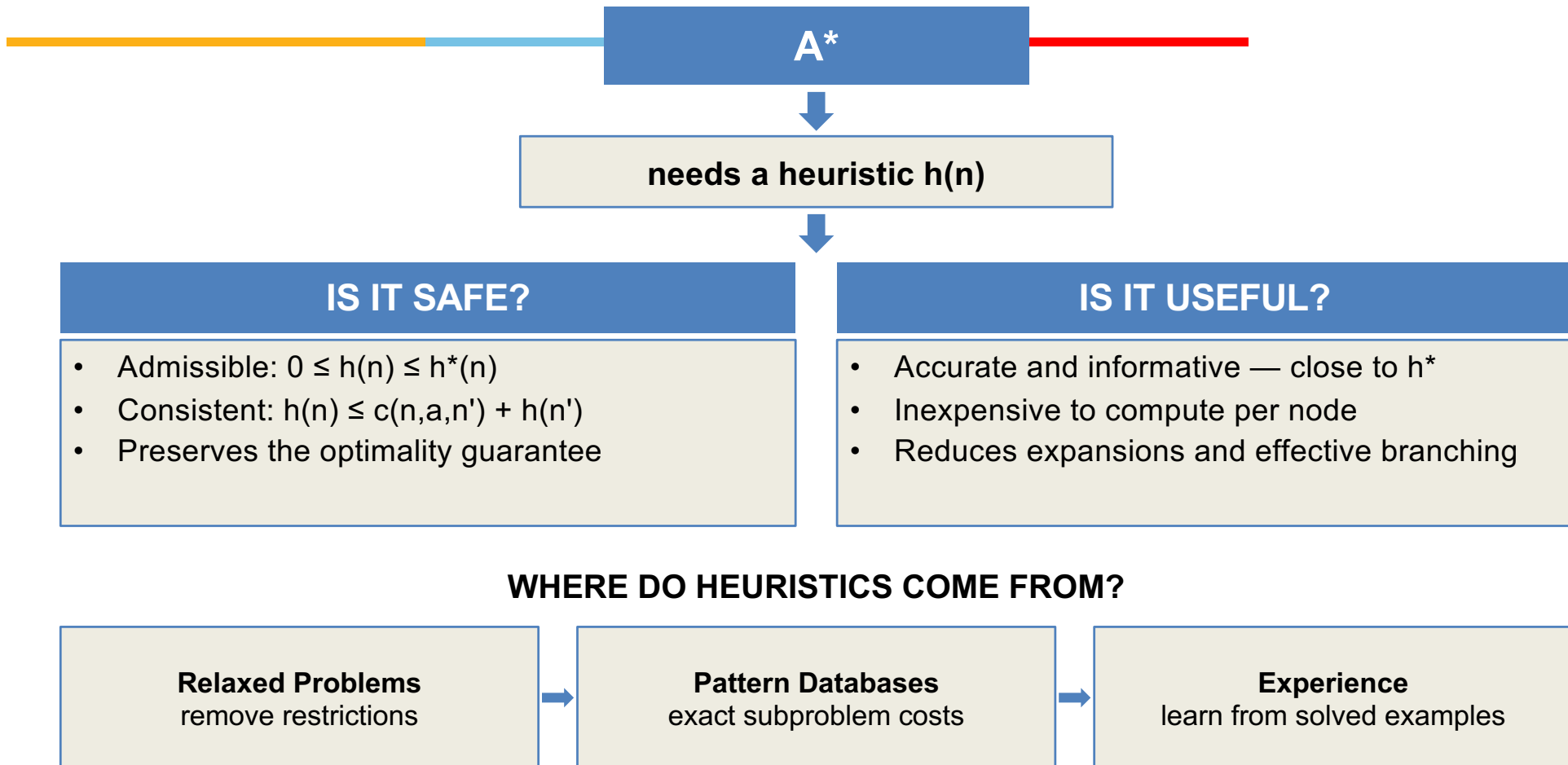
never exceeds the true remaining cost

better prediction ≠ guaranteed A* optimality

Key takeaway

Prediction quality and optimality guarantees are different questions. A fitted heuristic may overestimate on some states, and A* then loses its optimality guarantee.

One Big Picture



Lab 2 Connection



A* implementation — record the following for each run

Record

- $g(n)$, $h(n)$ and $f(n) = g(n) + h(n)$
- Final solution cost
- Nodes expanded
- Maximum frontier size

Suggested comparison

Run the same instances twice: once with $h = 0$, once with Manhattan distance.

“Does the better heuristic reduce node expansion?”

Metric	$h = 0$	Manhattan
Final cost	—	—
Nodes expanded	—	—
Maximum frontier	—	—

Both runs should return the same final cost: $h = 0$ and Manhattan distance are both admissible.



Common Mistakes and Exam Points

Do not confuse

$g(n)$

cost already incurred from the start

$h(n)$

estimated remaining cost to a goal

$f(n)$

$g(n) + h(n)$: estimated total cost

Remember

- Admissible = never overestimates
- Consistent = triangle inequality on every transition
- Consistency $\Rightarrow$ admissibility (not the reverse)

- A stronger admissible heuristic usually means fewer expansions
- Relaxed problem = easier problem giving a lower bound
- A learned heuristic may not preserve optimality

Tree-search A* needs admissibility; graph-search A* needs consistency; $h = 0$ gives UCS.

Exit Check



Answer these six questions before you leave — about 5 minutes.

1

Why is $h(n) = 0$ admissible but weak?

2

What is the difference between admissibility and consistency?

3

Why does Manhattan distance dominate misplaced tiles?

4

Why does solving a relaxed problem give an admissible heuristic?

5

What does a pattern database store?

6

Why can a learned heuristic break A*'s optimality guarantee?

If any answer is unclear, revisit Slides 6, 10, 11, 18, 21 and 25.

CS4 Summary and CS5 Bridge

CS4

“Use a heuristic to guide path search”



CS5

“Search directly through candidate solutions”

Coming next in CS5

Hill climbing



Simulated annealing



Local beam search



Optimisation
landscapes

Final takeaway: judge every heuristic from two perspectives

1. Is it safe?

admissible, and consistent for graph search

2. Is it useful?

informative and reasonably inexpensive to compute

Reference: T1 §3.5.2 (optimality of A*) and §3.6 (heuristic functions)